Lecture 25: MIPS Branch, Comparison & Jump Instructions (`beq`, `bne`, `slt`, `slti`, `j`, `jal`, `jr`)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand program control flow.
- Compare values using `slt` and `slti`.
- Use `beq` and `bne` for conditional branching.
- Use `j` for unconditional jumps.
- Understand `jal` and `jr` for function calls and returns.
- Implement `if`, `if-else`, and simple decision-making in MIPS.
- Debug branch instructions using the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how MIPS programs make decisions using `beq`, `bne`, `slt`, `slti`, `j`, `jal`, and `jr`, and implement `if` and `if-else` statements with practical examples in the MARS simulator."**

In the previous lecture, we learned how to load data from memory, perform calculations, and store the results back into memory.

But real-world programs don't always execute every statement one after another. Sometimes we need to make decisions.

For example:

- If a student's marks are greater than or equal to 50, display "Pass".
- Otherwise, display "Fail".

In high-level languages, we use `if`, `else`, and loops.

In MIPS Assembly Language, we achieve the same behavior using comparison, branch, and jump instructions.

Today, we'll learn how the processor changes the flow of execution using `beq`, `bne`, `slt`, `slti`, `j`, `jal`, and `jr`.

Let's begin.

---

2. Sequential Execution

By default, the CPU executes instructions one after another.

Instruction 1

↓

Instruction 2

↓

Instruction 3

↓

Instruction 4

This is called **sequential execution**.

Sometimes we need to **skip instructions** or **jump to another location**.

---

3. Control Flow Instructions

MIPS uses three categories:

Category	Instructions
Comparison	`slt`, `slti`
Conditional Branch	`beq`, `bne`
Unconditional Jump	`j`, `jal`, `jr`

---

4. Labels

Branch and jump instructions use **labels**.

Example:

loop:

exit:

Labels mark a location in the program.

---

5. The `beq` Instruction

Purpose

Branch if Equal.

Syntax

beq register1, register2, label

Meaning:

If register1 == register2

Jump to label.

Example

li $t0, 10

li $t1, 10

beq $t0, $t1, Equal

li $v0, 10

syscall

Equal:

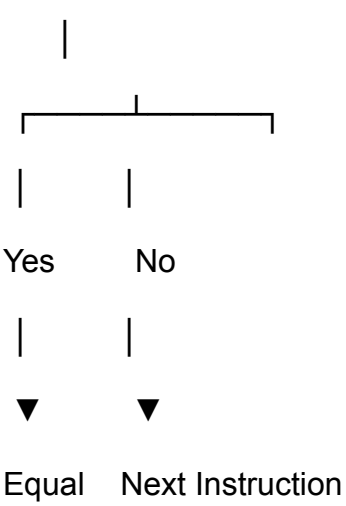
li $v0, 10

syscall

Since `$t0 == $t1`, execution jumps to `Equal`.

---

6. Flow Diagram



---

7. The bne Instruction

Purpose

Branch if Not Equal.

Syntax

bne register1, register2, label

Meaning:

If register1 ≠ register2

Jump to label.

Example

li $t0, 20

li $t1, 10

bne $t0, $t1, Different

Execution jumps because the values are different.

---

8. Comparison Using `slt`

MIPS has no direct instruction like:

if(a < b)

Instead, we use `slt`.

Syntax

slt destination, source1, source2

Meaning:

If source1 < source2

destination = 1

Otherwise

destination = 0

Example

li $t1, 5

li $t2, 10

slt $t0, $t1, $t2

Result:

$t0 = 1

because:

5 < 10

Another example:

li $t1, 20

li $t2, 10

slt $t0, $t1, $t2

Result:

$t0 = 0

---

9. `slti`

Compares a register with an immediate value.

Syntax

slti destination, source, immediate

Example:

slti $t0, $t1, 100

Meaning:

If $t1 < 100

$t0 = 1

Otherwise:

$t0 = 0

---

10. Implementing an `if` Statement

High-Level Language:

```
if(a==b)
{
    c=10;
}
```

MIPS:

beq $t0, $t1, Equal

j EndIf

Equal:

li $t2,10

EndIf:

Explain how the jump skips the `if` block when the condition is false.

---

11. Implementing `if-else`

C++:

```
if(a>b)
{
   max=a;
}
else
{
   max=b;
}
```

MIPS:

slt $t3,$t1,$t0

beq $t3,$zero,Else

move $t2,$t0

j End

Else:

move $t2,$t1

End:

This stores the larger value in `$t2`.

---

12. The `j` Instruction

Purpose

Unconditional Jump.

Syntax

j label

Example:

j Finish

Execution immediately continues at `Finish`.

---

13. The `jal` Instruction

Purpose

Jump And Link.

Used for calling functions.

Example:

jal Sum

What happens?

1. Stores the return address in `$ra`.
2. Jumps to `Sum`.

---

14. The `jr` Instruction

Purpose

Jump Register.

Usually used to return from a function.

Example:

jr $ra

The processor jumps back to the instruction after the `jal`.

Mention that functions and procedures will be covered in detail later.

---

15. Complete Example – Find the Larger Number

.text

.globl main

main:

li $t0,15

li $t1,20

slt $t2,$t1,$t0

beq $t2,$zero,Second

move $s0,$t0

j Finish

Second:

move $s0,$t1

Finish:

li $v0,10

syscall

After execution:

$s0 = 20

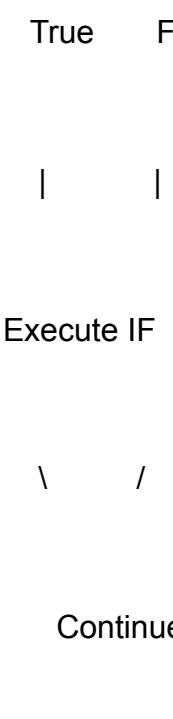
---

16. Decision-Making Flowchart

Compare

Condition?

/ \



---

17. Real-Life Analogy

Imagine a traffic signal.

Green

↓

Go

Red

↓

Stop

Yellow

↓

Slow Down

The driver makes decisions based on conditions, just as a program changes its execution path using branch instructions.

---

18. Common Mistakes

Students often:

❌ Forget labels.

Every branch destination must have a label.

❌ Use `beq` for less-than comparisons.

Use:

slt

first, then branch.

❌ Forget to jump after the `if` block.

This causes the `else` block to execute even when the `if` condition is true.

❌ Confuse `j` and `jal`.

- `j` only jumps.
- `jal` jumps **and saves the return address** in `$ra`.

🎤 Lecture Summary

Today, we learned how MIPS Assembly Language changes the flow of a program using comparison, branch, and jump instructions.

We explored `beq`, `bne`, `slt`, `slti`, `j`, `jal`, and `jr`, and used them to implement `if` statements, `if-else` logic, and simple decision-making. We also introduced function calls and returns, which will be studied in greater depth later in the course.

In the next lecture, we'll build on these concepts by learning **loops in MIPS Assembly Language**, including `while`, `for`, and `do-while` loops. We'll write practical programs that print sequences, calculate sums, and iterate through arrays using branch and jump instructions.

Thank you for watching, and I'll see you in the next lecture.
